

vaHera and Partition-Coordinate Refinement Types: A Type System for Symbolic Physical Derivations and Selection-Rule Enforcement

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
AIME Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

May 31, 2026

Abstract

We introduce a refinement-type system over the vaHera abstract syntax tree (AST) in which partition coordinates (n, ℓ, m, s) and the selection rules $\Delta\ell = \pm 1$, $\Delta m \in \{0, \pm 1\}$, $\Delta s = 0$ are first-class type-theoretic predicates rather than runtime constraints. The vaHera AST consists of four frozen variants—Literal, Call, Compose, and Hole—and provides the symbolic substrate over which physical derivations expressing transitions between partition labels are written. We prove three principal results. (i) The *Compositionality Lemma*: refinement types compose under Compose via predicate conjunction, with no loss of decidability. (ii) The *Forbidden-Transition Theorem*: a vaHera fragment whose Compose chain implies a transition violating any registered selection rule is rejected by typecheck at static time; the rejection is structural, not probabilistic, and requires no run-time observation. (iii) The *Decidability Theorem*: refinement-type checking under a finite selection-rule set is decidable in $\mathcal{O}(|F| \cdot |\sigma|)$ where $|F|$ is the syntactic size of the fragment and $|\sigma|$ is the rule-set cardinality, and the checker is a total function. We provide a complete inference-rule system, soundness and completeness proofs, and a worked example illustrating the rejection of a fragment encoding an $\ell = 0 \rightarrow \ell = 2$ transition. The framework is purely syntactic: every theorem is provable from the four vaHera variants, the partition-label invariants, and the algebra of selection-rule predicates, without recourse to physical postulates

or runtime evaluation. We outline how the type system serves as the formal substrate for physical-derivation languages that compile to vaHera, including the temporal-programming Tempus calculus, partition-mechanics expressions over bounded Hamiltonian systems, and detector-classification predicates in real-time event-selection systems.

Keywords: refinement types, partition coordinates, selection rules, vaHera, symbolic computation, static verification, type-directed rejection, compositionality, physical derivation, categorical type theory.

Contents

I	Preliminaries	2
1	Introduction	2
1.1	Motivation	2
1.2	The vaHera substrate	3
1.3	Selection rules as type predicates	3
1.4	Overview of main results	3
2	The vaHera AST	4
2.1	Syntax	4
2.2	Values and base types	4
2.3	Operations	4
2.4	Standard typing judgement	4
II	Partition Coordinates as Refinement Base Types	4

3	Partition Labels	4	9.5	Three-Route Verification	11
3.1	Definition	4			
3.2	Label cardinality	5			
3.3	Label projection	5	VII	Discussion	11
4	Refinement Types over Partition Labels	5	10	Comparison with Existing Refinement-Type Systems	11
4.1	Refinement formation	5	10.1	Liquid Types	11
4.2	Selection rules	6	10.2	Refinement Types for Haskell . . .	11
4.3	Operations with refinement	6	10.3	ML-Style Refinements	11
			10.4	Dependent Types	12
			10.5	Hoare Logic	12
III	Refinement Typing	6	11	Limitations and Open Questions	12
5	Refined Typing Judgement	6	11.1	Predicate language	12
5.1	Definition	6	11.2	Hole substitution	12
5.2	The Compositionality Lemma . . .	6	11.3	Operation registration	12
5.3	The selection-rule refinement convention	7	11.4	Selection-rule learning	12
6	Soundness and Completeness	7	12	Conclusion	12
6.1	Operational semantics	7			
6.2	Soundness	7			
6.3	Completeness	7			
6.4	The Forbidden-Transition Theorem	8			
IV	Decidability and Algorithms	8	Part I		
7	Decidability	8	Preliminaries		
7.1	Complexity of typecheck	8	1	Introduction	
7.2	The typecheck algorithm	9	1.1	Motivation	
7.3	The role of the bijection Φ	9			
V	Worked Example	9			
8	The $\ell = 0 \rightarrow \ell = 2$ Rejection	9			
8.1	Setup	9			
8.2	Direct composition: rejected	10			
8.3	Through-route composition: also rejected	10			
8.4	Significance	10			
VI	Connections to Applications	10			
9	Applications across the Symbolic-Physics Substrate	10			
9.1	Bounded Hamiltonian Systems . .	10			
9.2	Mass-Spectrometric Trajectories .	10			
9.3	Temporal Programming and Cell Registries	11			
9.4	Real-Time Event-Selection Systems	11			

1 Introduction

1.1 Motivation

Symbolic representations of physical derivations are everywhere: Feynman diagrams encode interaction amplitudes; partition-state transition graphs encode bounded Hamiltonian dynamics; cell-registry programs encode real-time event-selection logic; and ion-trajectory equations encode mass-spectrometric measurement. In every case, the symbolic representation must obey constraints that arise from the underlying categorical structure of partition coordinates—in particular, from the *selection rules* that determine which transitions between partition labels are admissible. Without a type-level mechanism for enforcing such rules, the symbolic substrate admits expressions that have no physical interpretation and that, if executed, produce well-formed but operationally meaningless results.

The standard response to this problem is to detect ill-formed expressions at runtime: the dispatch path evaluates a fragment and rejects results that fail to satisfy the relevant constraints. This response is unsatisfactory in three ways.

First, it requires actual execution before rejection, which is prohibitively expensive for high-throughput dispatch substrates such as kernels driving real-time detector systems. Second, it conflates two distinct categories of failure—those arising from runtime conditions (resource exhaustion, network unavailability) and those arising from structural ill-formedness—producing diagnostics that cannot be attributed to the right cause. Third, and most fundamentally, it forfeits the structural certainty available at compile time: a fragment whose composition chain implies a forbidden transition is ill-formed as a syntactic object, not merely as a runtime computation.

This paper proposes that the right response is to lift selection-rule enforcement into the type system. We define a refinement-type discipline over the vaHera AST in which partition labels are types and selection rules are refinement predicates. A vaHera fragment is well-typed if and only if every Compose-stage transition respects every registered selection rule; type-checking is decidable in linear time in the fragment size; and the typing relation is structurally compositional.

1.2 The vaHera substrate

The vaHera AST is a four-variant inductive datatype originally introduced as the symbolic interchange format between resolvers and providers in the Purpose framework (Sachikonye, 2026c). The four variants are:

- $\text{Literal}(v)$, the embedding of a runtime value;
- $\text{Call}(\mathcal{O}, \overline{F})$, the invocation of a named operation $\mathcal{O}$ with argument fragments;
- $\text{Compose}(\overline{F})$, the sequential composition of fragments with carry threading;
- $\text{Hole}(x)$, a typed placeholder awaiting substitution.

The variants are frozen: no version of the framework adds, removes, or renames them, and downstream consumers may rely on their permanence (per the stability contract in Sachikonye, 2026c §2.1). This stability is what permits a refinement-type system to be defined once and to apply uniformly across all clients of the AST.

1.3 Selection rules as type predicates

The central technical move of this paper is to interpret selection rules—binary predicates on pairs of partition labels asserting that a given transition is admissible—as refinement predicates

over partition-coordinate types. In standard refinement-type discipline (Rondon et al., 2008; Vazou et al., 2014; Freeman and Pfenning, 1991), a refinement type $\{x : B \mid \phi(x)\}$ is the subset of values of base type B satisfying predicate ϕ . Our generalisation is mild: we replace the single-value predicate $\phi(x)$ with a transition predicate $\sigma(x_1, x_2)$ relating the input and output labels of an operation. This allows a single operation type signature

$$\mathcal{O} : \{x_1 : \mathcal{L} \mid \text{true}\} \rightarrow \{x_2 : \mathcal{L} \mid \sigma(x_1, x_2)\}$$

to express *any transition admissible under σ* , with σ ranging over an arbitrary finite set of selection rules. Composition then accumulates the conjunction of the per-stage transition predicates, and the Compositionality Lemma (Theorem 5.2) ensures the accumulated predicate is itself decidable.

1.4 Overview of main results

The paper establishes the following:

1. **Theorem 3.3** (Label Formation): the four-tuple partition labels (n, ℓ, m, s) form a well-defined inductive type with closed-form bounds.
2. **Theorem 4.3** (Refinement Type Formation): refinements $\{x : B \mid \phi(x)\}$ over partition-label base types are well-formed and yield decidable subsets.
3. **Theorem 5.2** (Compositionality Lemma): for any $\text{Compose}(\overline{F})$, the refinement type of the composite equals the conjunction of per-stage refinement types under carry-threading semantics.
4. **Theorem 6.1** (Soundness of typecheck): every fragment accepted by typecheck produces, on execution, a Value whose label satisfies every registered selection rule.
5. **Theorem 6.2** (Completeness of typecheck): every fragment whose composition chain implies a transition violating any registered selection rule is rejected by typecheck.
6. **Theorem 6.3** (Forbidden-Transition Theorem): the principal consequence: typecheck rejection of selection-rule-violating fragments is structural, not probabilistic, and occurs at static time with no runtime observation required.
7. **Theorem 7.1** (Decidability): refinement-type checking under finite σ is decidable in $\mathcal{O}(|F| \cdot |\sigma|)$.

2 The vaHera AST

2.1 Syntax

Definition 2.1 (vaHera AST). *The vaHera AST is the inductive datatype F defined by the grammar*

$$F ::= \text{Literal}(v) \mid \text{Call}(\mathcal{O}, \overline{F}) \mid \text{Compose}(\overline{F}) \mid \text{Hole}(x)$$

where v ranges over runtime Values, $\mathcal{O}$ over operation names, $\overline{F}$ over finite sequences of fragments (the Call case carries a named-argument map, and the Compose case carries an ordered list), and x over identifiers.

Remark 2.2. *The four variants are pairwise disjoint and exhaustive over the AST. Every well-formed fragment is, syntactically, exactly one of these four shapes. This is the frozen surface from [Sachikonye \(2026c\)](#) §2.1.*

2.2 Values and base types

Definition 2.3 (Value). *The runtime Value space V is the inductive datatype*

$$v ::= \text{Null} \mid \text{Bool}(b) \mid \text{Num}(r) \mid \text{Str}(s) \\ \mid \text{List}(\overline{v}) \mid \text{Record}(\overline{(k, v)})$$

where $b \in \mathbb{B}$, $r \in \mathbb{R}$, s is a finite character string, $\overline{v}$ is a finite sequence of Values, and $\overline{(k, v)}$ is a finite map from string keys to Values.

Definition 2.4 (Base type). *A base type B is one of $\{\text{Str}, \text{Num}, \text{Bool}, \text{Unit}\}$ or a parameterised constructor $\text{List}(B')$, a Named type $\text{Named}(\tau)$ for a type name τ , or a type variable $\text{Var}(\alpha)$. The set of base types is denoted $\mathcal{B}$.*

The base-type discipline is the standard Hindley-Milner-style typing layer ([Hindley, 1969](#); [Milner, 1978](#); [Damas and Milner, 1982](#)). Every Value has a most-general base type, and the typing judgement $\vdash v : B$ is decidable in linear time.

2.3 Operations

Definition 2.5 (Operation). *An operation $\mathcal{O}$ is a record*

$$\mathcal{O} = \langle \text{name}, \text{inputs}, \text{output}, \text{desc} \rangle$$

where *name* is a string identifier, *inputs* is a finite map from named argument keys to base

types, *output* is a base type, and *desc* is a human-readable description. The set of operations registered in a given context is the operation registry $\mathcal{R}$.

Remark 2.6. *The desc field carries no semantic content for the type system. We retain it for compatibility with the runtime substrate, which uses it for introspection.*

2.4 Standard typing judgement

The standard (non-refinement) typing judgement $\Gamma \vdash F : B$ asserts that under typing context Γ , the fragment F has base type B .

Definition 2.7 (Standard typing). *The typing relation $\Gamma \vdash F : B$ is the least relation closed under the rules:*

$$\frac{\vdash v : B}{\Gamma \vdash \text{Literal}(v) : B} (T\text{-LIT}) \quad \frac{\Gamma(x) = B}{\Gamma \vdash \text{Hole}(x) : B} (T\text{-HOL})$$

$$\frac{\mathcal{O} = \langle _, \overline{(k_i, B_i)}, B, _ \rangle \in \mathcal{R} \quad \forall i : \Gamma \vdash F_i : B_i}{\Gamma \vdash \text{Call}(\mathcal{O}, \overline{(k_i, F_i)}) : B} (T\text{-CALL})$$

$$\frac{\Gamma \vdash F_1 : B_1 \quad \dots \quad \Gamma \vdash F_n : B_n \quad \forall i \geq 2 : B_{i-1} \succeq \text{in}_i}{\Gamma \vdash \text{Compose}(\overline{F}) : B_n} (T\text{-COM})$$

where in_i is the carry-input type of stage i (the type of the implicit **input** argument), and $\succeq$ is the standard subtyping relation.

Proposition 2.8 (Standard typing is decidable). *For every Γ and every well-formed F , the judgement $\Gamma \vdash F : B$ is decidable in linear time in $|F|$.*

Proof. Straightforward structural induction over F : each rule has a unique conclusion shape, premises decrease in fragment size, and base-type subtyping is decidable in constant time per check. $\square$ $\square$

The standard typing judgement is the foundation on which we layer refinement.

Part II

Partition Coordinates as Refinement Base Types

3 Partition Labels

3.1 Definition

Definition 3.1 (Partition Label). *A partition label is a 4-tuple*

$$\lambda = (n, \ell, m, s) \in \mathbb{N} \times \mathbb{N} \times \mathbb{Z} \times \{-\frac{1}{2}, +\frac{1}{2}\}$$

subject to the constraints

$$n \geq 1, \quad (1)$$

$$0 \leq \ell \leq n - 1, \quad (2)$$

$$-\ell \leq m \leq +\ell. \quad (3)$$

The set of all partition labels is denoted $\mathcal{L}$.

Remark 3.2. *The constraints (1)–(3) are the nested boundary conditions of three-dimensional bounded oscillatory systems, derived from first principles in Sachikonye (2026a). We adopt them here as axiomatic for the type system; the question of how they arise physically is orthogonal to the present development.*

3.2 Label cardinality

Theorem 3.3 (Label Formation). *The set $\mathcal{L}$ of partition labels is well-defined and admits the following cardinality structure:*

1. *For each $n \geq 1$, the number of labels with that principal value is $C(n) = 2n^2$.*
2. *The cumulative count of labels with principal value at most $n_{\max}$ is $N(n_{\max}) = n_{\max}(n_{\max} + 1)(2n_{\max} + 1)/3$.*
3. *There exists a canonical bijection $\Phi : \mathbb{N}^+ \rightarrow \mathcal{L}$ assigning each positive integer to a unique label.*

Proof. (i) At principal level n , ℓ ranges over $\{0, 1, \dots, n - 1\}$ (by (2)); for each ℓ , m ranges over $2\ell + 1$ values (by (3)); and s contributes a factor of 2. Summing:

$$C(n) = 2 \sum_{\ell=0}^{n-1} (2\ell + 1) = 2(n^2 - n) + 2n = 2n^2.$$

(ii) Applying $\sum_{n=1}^N n^2 = N(N + 1)(2N + 1)/6$ with the factor of 2 from (i):

$$N(n_{\max}) = \sum_{n=1}^{n_{\max}} 2n^2 = \frac{n_{\max}(n_{\max} + 1)(2n_{\max} + 1)}{3}.$$

(iii) Define $\Phi(M)$ by determining $n = \lfloor \sqrt{M/2} \rfloor + 1$ via inversion of (ii), then enumerating (ℓ, m, s) lexicographically within level n . The construction is total and injective; surjectivity follows from $\bigsqcup_n \{(n, \ell, m, s) : 0 \leq \ell < n, |m| \leq \ell, s \in \{\pm 1/2\}\} = \mathcal{L}$. $\square$

Remark 3.4. *Theorem 3.3 (iii) gives an effective procedure for label enumeration: every positive integer indexes a unique label, and every label has a unique integer index. We will exploit this bijection in Section 7 to obtain a constant-time membership test.*

3.3 Label projection

For a label $\lambda = (n, \ell, m, s)$, we write $\pi_n(\lambda) = n$, $\pi_\ell(\lambda) = \ell$, $\pi_m(\lambda) = m$, $\pi_s(\lambda) = s$ for the four canonical projections, and $\Delta : \mathcal{L} \times \mathcal{L} \rightarrow \mathbb{Z}^4$ for the componentwise difference $\Delta(\lambda_1, \lambda_2) = (n_2 - n_1, \ell_2 - \ell_1, m_2 - m_1, s_2 - s_1)$.

4 Refinement Types over Partition Labels

4.1 Refinement formation

Definition 4.1 (Refinement Type). *A refinement type over base type $B \in \mathcal{B}$ with predicate ϕ is the pair*

$$\{x : B \mid \phi(x)\}$$

where ϕ is a computable Boolean-valued function on values of base type B . The set of refinement types is denoted $\mathcal{R}$.

The base-type refinements in standard literature (Rondon et al., 2008; Vazou et al., 2014) are single-value predicates of the form $\phi(x)$. Our development requires a generalisation: predicates that relate *pairs* of labels, the input and output of a transition.

Definition 4.2 (Transition Refinement). *A transition refinement is a refinement of the form*

$$\{x_2 : B_2 \mid \sigma(x_1, x_2)\}$$

where x_1 is the input value of a transition and x_2 is the output value, and $\sigma : \mathcal{L} \times \mathcal{L} \rightarrow \mathbb{B}$ is a binary predicate.

Theorem 4.3 (Refinement Type Formation). *For every base type $B \in \mathcal{B}$ and every computable predicate ϕ , the refinement type $\{x : B \mid \phi(x)\}$ is well-formed. Membership in $\{x : B \mid \phi(x)\}$ is decidable in time $\mathcal{O}(T_\phi)$, where T_ϕ is the cost of evaluating ϕ on a single value.*

Proof. Membership: $v \in \{x : B \mid \phi(x)\}$ iff $\vdash v : B$ and $\phi(v) = \text{true}$. The first conjunct is decidable in linear time by Proposition 2.8; the second conjunct is decidable in $\mathcal{O}(T_\phi)$ by hypothesis. The conjunction is therefore decidable in $\mathcal{O}(|v| + T_\phi)$. Well-formedness follows from $B \in \mathcal{B}$ being a base type and ϕ being computable. $\square$ $\square$

4.2 Selection rules

Definition 4.4 (Selection Rule). *A selection rule is a predicate $\sigma \in \mathcal{L} \times \mathcal{L} \rightarrow \mathbb{B}$ asserting that a transition from λ_1 to λ_2 is admissible (true) or forbidden (false).*

Example 4.5 (Standard physical selection rules). *The following are commonly used:*

- *Electric dipole (E1):* $\sigma_{E1}(\lambda_1, \lambda_2) \iff \pi_\ell(\lambda_2) - \pi_\ell(\lambda_1) \in \{-1, +1\}$ and $\pi_m(\lambda_2) - \pi_m(\lambda_1) \in \{-1, 0, +1\}$.
- *Magnetic dipole (M1):* $\sigma_{M1}(\lambda_1, \lambda_2) \iff \pi_\ell(\lambda_2) = \pi_\ell(\lambda_1)$ and $\pi_m(\lambda_2) - \pi_m(\lambda_1) \in \{-1, 0, +1\}$.
- *Spin conservation:* $\sigma_{\text{spin}}(\lambda_1, \lambda_2) \iff \pi_s(\lambda_2) = \pi_s(\lambda_1)$.
- *Principal advancement:* $\sigma_{\Delta n}(\lambda_1, \lambda_2) \iff \pi_n(\lambda_2) \geq \pi_n(\lambda_1)$.

We emphasise that the type system is parametric in σ : any computable binary predicate yields a selection rule, and the framework imposes no preference among rule sets.

Definition 4.6 (Selection-Rule Registry). *A selection-rule registry is a finite set $\sigma_\star = \{\sigma_1, \dots, \sigma_k\}$ of selection rules. A transition (λ_1, λ_2) is admissible under $\sigma_\star$ iff $\bigwedge_{i=1}^k \sigma_i(\lambda_1, \lambda_2)$.*

The registry is the unit of policy: changing the registered rules changes which transitions are admissible. For the remainder of the paper, we fix an arbitrary $\sigma_\star$.

4.3 Operations with refinement

Definition 4.7 (Refined Operation). *A refined operation extends Definition 2.5 with input and output refinements:*

$$\mathcal{O}^R = \langle \text{name}, \overline{(k_i, B_i, \phi_i^{\text{in}})}, B, \phi^{\text{out}}, \text{desc} \rangle$$

where ϕ_i^{in} is the refinement predicate on the i -th input and ϕ^{out} is the refinement predicate on the output. The set of refined operations is $\mathcal{R}^+$.

The output refinement may depend on the input values; in the partition-label setting, $\phi^{\text{out}}(\lambda_2) = \sigma_\star(\lambda_1^{\text{in}}, \lambda_2)$ for a designated input λ_1^{in} carrying the source label.

Part III

Refinement Typing

5 Refined Typing Judgement

5.1 Definition

The refined typing judgement $\Gamma \vdash F : \{x : B \mid \phi(x)\}$ extends Definition 2.7.

Definition 5.1 (Refined typing). *The refined typing relation is closed under the rules*

$$\frac{\vdash v : B \quad \phi(v) = \text{true}}{\Gamma \vdash \text{Literal}(v) : \{x : B \mid \phi(x)\}} (R\text{-LIT})$$

$$\frac{\Gamma(x) = \{y : B \mid \phi(y)\}}{\Gamma \vdash \text{Hole}(x) : \{y : B \mid \phi(y)\}} (R\text{-HOL})$$

$$\frac{\mathcal{O}^R \in \mathcal{R}^+ \quad \forall i : \Gamma \vdash F_i : \{y_i : B_i \mid \phi_i^{\text{in}}(y_i)\}}{\Gamma \vdash \text{Call}(\mathcal{O}^R, \overline{(k_i, F_i)}) : \{x : B \mid \phi^{\text{out}}(x, \overline{y_i})\}} (R\text{-CALL})$$

$$\frac{\Gamma \vdash F_1 : \{y_1 : B_1 \mid \phi_1\} \quad \dots \quad \Gamma \vdash F_n : \{y_n : B_n \mid \phi_n\}}{\Gamma \vdash \text{Compose}(\overline{F}) : \{x : B_n \mid \bigwedge_{i=1}^n \phi_i \# \text{thread}\}} (R\text{-COM})$$

where $\# \text{thread}$ denotes that the carry value of stage i is the input value y_{i+1}^{in} of stage $i + 1$.

The carry-threading premise of (R-COMP) is what makes the rule non-trivial: the output refinement of one stage is fed into the input refinement of the next, accumulating constraints as the composition chain grows.

5.2 The Compositionality Lemma

Theorem 5.2 (Compositionality Lemma). *Let $F = \text{Compose}(F_1, F_2, \dots, F_n)$ be a Compose chain. For all i , let*

$$\Gamma \vdash F_i : \{y_i : B_i \mid \phi_i(y_{i-1}, y_i)\}$$

under the carry-threading semantics, with y_0 a designated initial value. Then

$$\Gamma \vdash F : \left\{ x : B_n \mid \bigwedge_{i=1}^n \phi_i(y_{i-1}, y_i) \right\}.$$

Proof. By structural induction on n .

Base case ($n = 1$): $F = \text{Compose}(F_1)$. The hypothesis gives $\Gamma \vdash F_1 : \{y_1 : B_1 \mid \phi_1(y_0, y_1)\}$, and (R-COMP) with $n = 1$ reduces to the same judgement (the conjunction over a single conjunct is the conjunct itself). The result holds.

Inductive case ($n = k + 1$, assuming the result for $n = k$): Let $F^{(k)} = \text{Compose}(F_1, \dots, F_k)$ and $F = \text{Compose}(F^{(k)}, F_{k+1})$. By the inductive hypothesis,

$$\Gamma \vdash F^{(k)} : \{y_k : B_k \mid \Psi_k(y_0, y_1, \dots, y_k)\}$$

where $\Psi_k = \bigwedge_{i=1}^k \phi_i$. By (R-COMP) applied to the binary case (two-stage compose),

$$\Gamma \vdash \text{Compose}(F^{(k)}, F_{k+1}) : \{x : B_{k+1} \mid \Psi_k \wedge \phi_{k+1}(y_k, y_{k+1})\} \text{ and } \phi(v) = \text{true}.$$

Since $\Psi_k \wedge \phi_{k+1} = \bigwedge_{i=1}^{k+1} \phi_i$, the inductive step holds.

By induction, the result holds for all $n \geq 1$. $\square$

Corollary 5.3 (Conjunction-Preservation). *The refinement type of a Compose chain is the conjunction of the per-stage refinements modulo carry-threading. In particular, if any single ϕ_i is unsatisfiable, the type of the composite is uninhabited.*

Proof. Direct from Theorem 5.2: if $\phi_i \equiv \text{false}$, then $\bigwedge_j \phi_j \equiv \text{false}$, and no value satisfies false. $\square$

5.3 The selection-rule refinement convention

For the remainder of the paper, we work under the *selection-rule refinement convention*: every refined operation $\mathcal{O}^R$ whose input and output are partition-label types has output refinement

$$\phi^{\text{out}}(\lambda_2, \lambda_1) = \bigwedge_{\sigma \in \sigma_*} \sigma(\lambda_1, \lambda_2),$$

i.e. the operation's output is well-typed iff the implied transition respects every registered selection rule. The convention is a notational specialisation; the underlying type system is unchanged.

6 Soundness and Completeness

6.1 Operational semantics

To state soundness, we recall the standard operational semantics of vaHera (Sachikonye, 2026c). The dispatch evaluation $\text{eval}(F) \in V \cup \{\perp\}$ is defined by structural recursion:

$$\text{eval}(\text{Literal}(v)) = v$$

$$\text{eval}(\text{Call}(\mathcal{O}, \overline{(k_i, F_i)})) = \mathcal{O}(\overline{(k_i, \text{eval}(F_i))})$$

$$\text{eval}(\text{Compose}(F_1, \dots, F_n)) = \text{thread}(\text{eval}(F_1), \dots, \text{eval}(F_n))$$

$$\text{eval}(\text{Hole}(x)) = \perp$$

where thread pipes the output of each stage into the implicit input of the next, and $\perp$ denotes the failure to dispatch (unresolved Hole).

6.2 Soundness

Theorem 6.1 (Soundness of typecheck). *If $\Gamma \vdash F : \{x : B \mid \phi(x)\}$ and $\text{eval}(F) = v \neq \perp$, then*

$$\phi(v) = \text{true}.$$

Proof. By induction on the derivation of $\Gamma \vdash F : \{x : B \mid \phi(x)\}$.

Case (R-Lit): $F = \text{Literal}(v)$. The premises give $\vdash v : B$ and $\phi(v) = \text{true}$. The conclusion follows by $\text{eval}(\text{Literal}(v)) = v$.

Case (R-Hol): $F = \text{Hole}(x)$. Then $\text{eval}(F) = \perp$, contradicting the hypothesis $\text{eval}(F) \neq \perp$. The case is vacuous.

Case (R-Call): $F = \text{Call}(\mathcal{O}^R, \overline{(k_i, F_i)})$. By the inductive hypothesis, each $\text{eval}(F_i)$ is a value of type B_i satisfying ϕ_i^{in} . The operation $\mathcal{O}^R$ is, by Definition 4.7, refined: its output, computed as $\mathcal{O}(\overline{(k_i, \text{eval}(F_i))})$, has type B and satisfies ϕ^{out} . The conclusion follows.

Case (R-Comp): $F = \text{Compose}(F_1, \dots, F_n)$. By Theorem 5.2, the type of F is $\{x : B_n \mid \bigwedge_i \phi_i\}$. By the inductive hypothesis applied to each F_i , each intermediate value $v_i = \text{eval}(F_i)$ has type B_i and satisfies ϕ_i . The final value $v_n = \text{thread}(v_1, \dots, v_n)$ inherits type B_n from the last stage and satisfies $\bigwedge_i \phi_i$ by the threading invariant.

The induction is complete. $\square$ $\square$

6.3 Completeness

Theorem 6.2 (Completeness of typecheck). *If F is a Compose chain whose thread semantics implies a transition $(\lambda_i, \lambda_{i+1})$ for some i that violates some registered selection rule $\sigma \in \sigma_*$, then $\text{typecheck}(F)$ fails: there is no derivation $\Gamma \vdash F : T$ for any refinement type T under the selection-rule refinement convention.*

Proof. Suppose, for contradiction, that $\Gamma \vdash F : \{x : B \mid \phi(x)\}$ for some ϕ . By Theorem 5.2, $\phi = \bigwedge_{j=1}^n \phi_j$ where ϕ_j is the refinement of stage j in the Compose chain.

By the selection-rule refinement convention, stage i (the stage whose transition is forbidden) has refinement $\phi_i(\lambda_i, \lambda_{i+1}) = \bigwedge_{\sigma \in \sigma_*} \sigma(\lambda_i, \lambda_{i+1})$. By hypothesis, some $\sigma(\lambda_i, \lambda_{i+1}) = \text{false}$. Hence $\phi_i = \text{false}$ at the relevant transition.

By Corollary 5.3, the conjunction $\bigwedge_j \phi_j$ is unsatisfiable; the type $\{x : B \mid \bigwedge_j \phi_j\}$ is uninhabited.

By Soundness (Theorem 6.1), an inhabited refinement type would be required for a successful eval producing a non- $\perp$ value. The uninhabited type forces $\text{eval}(F) = \perp$ (the fragment has no well-typed evaluation). But this contradicts the assumption that typecheck accepted F —typecheck requires inhabitation. $\square$ $\square$

6.4 The Forbidden-Transition Theorem

We can now state the principal consequence.

Theorem 6.3 (Forbidden-Transition Theorem). *Under the selection-rule refinement convention with registry σ_* :*

1. *Every vaHera Compose chain F whose implied transitions all respect σ_* is accepted by typecheck (modulo standard typing requirements).*
2. *Every vaHera Compose chain F that implies a transition violating any $\sigma \in \sigma_*$ is rejected by typecheck at static time, with no runtime observation required.*

The acceptance/rejection is a decidable function of the syntactic structure of F and the predicates in σ_ .*

Proof. Combine Theorems 6.1 and 6.2: soundness gives (i), completeness gives (ii). Decidability of the joint judgement follows from each σ being computable and the Compose chain being finite. $\square$ $\square$

Remark 6.4 (Rejection is structural). *Theorem 6.3 (ii) is the key claim of this paper. It says that selection-rule-violating fragments are rejected for a structural reason—the refinement type is uninhabited—rather than because of a probabilistic event at runtime. The rejection occurs before dispatch, requires no execution, and is reproducible across implementations.*

Part IV

Decidability and Algorithms

7 Decidability

7.1 Complexity of typecheck

Theorem 7.1 (Decidability of typecheck). *Refinement-type checking of a vaHera fragment F under selection-rule registry σ_* runs in time $\mathcal{O}(|F| \cdot |\sigma_*|)$, where $|F|$ is the syntactic size of F and $|\sigma_*|$ is the cardinality of σ_* , assuming constant-time evaluation of each $\sigma \in \sigma_*$.*

Proof. We give a constructive algorithm.

Algorithm. Walk F once in pre-order. For each Compose node, examine the per-stage refinements pairwise: for the transition from stage i to stage $i + 1$, evaluate each $\sigma \in \sigma_*$ on the implied $(\lambda_i, \lambda_{i+1})$ pair. Reject if any σ returns false; else continue. For each Call node, validate the operation signature against the standard typing premises (Proposition 2.8, linear time). For Literal and Hole, return immediately.

Complexity. Walking F visits each AST node exactly once. At each Compose node with k stages, the per-stage refinement check costs $\mathcal{O}(|\sigma_*|)$ per transition for $k - 1$ transitions, totalling $\mathcal{O}(k \cdot |\sigma_*|)$. Summing over all Compose nodes gives $\mathcal{O}(N_C \cdot |\sigma_*|)$ where N_C is the total stage count, which is bounded by $|F|$. The standard typing component adds $\mathcal{O}(|F|)$. Total: $\mathcal{O}(|F| \cdot |\sigma_*|)$.

Termination. Pre-order walk is structurally recursive on a finite AST, hence terminating. $\square$ $\square$

Remark 7.2. *The constant-time assumption on $\sigma \in \sigma_*$ holds for all selection rules in Example 4.5: they are pure arithmetic comparisons on partition labels with bounded data. For more elaborate σ*

involving, say, table lookups, the bound generalises to $\mathcal{O}(|F| \cdot |\sigma_\star| \cdot T_{\max})$ where $T_{\max}$ is the maximum per-rule evaluation cost.

7.2 The typecheck algorithm

Algorithm 1 Refinement-type check

```

1: function CHECK( $F, \Gamma, \Sigma_\star$ )
2:   if  $F$  is a literal  $\text{Literal}(v)$  then
3:     return  $\vdash v : B$  for some base type  $B$ 
4:   else if  $F$  is a hole  $\text{Hole}(x)$  then
5:     return  $x \in \text{dom}(\Gamma)$ 
6:   else if  $F$  is a call  $\text{Call}(\Omega, \text{args})$  then
7:     require  $\Omega \in \mathcal{R}^+$ 
8:     for each named argument  $\text{args}[k_i] = F_i$  do
9:        $r_i \leftarrow \text{CHECK}(F_i, \Gamma, \Sigma_\star)$ 
10:      if  $r_i$  is false then
11:        return false
12:      end if
13:    end for
14:    return true
15:   else if  $F$  is a compose  $\text{Compose}(F_1, \dots, F_n)$  then
16:      $\lambda_{\text{prev}} \leftarrow \text{init}$ 
17:     for  $i = 1$  to  $n$  do
18:        $r_i \leftarrow \text{CHECK}(F_i, \Gamma, \Sigma_\star)$ 
19:       if  $r_i$  is false then
20:         return false
21:       end if
22:        $\lambda_i \leftarrow \text{labelOf}(F_i)$ 
23:       if  $i > 1$  then
24:         for each  $\sigma \in \Sigma_\star$  do
25:           if  $\sigma(\lambda_{\text{prev}}, \lambda_i)$  is false then
26:             return false
27:           end if
28:         end for
29:       end if
30:        $\lambda_{\text{prev}} \leftarrow \lambda_i$ 
31:     end for
32:     return true
33:   end if
34: end function

```

Proposition 7.3 (Algorithm correctness). *Algorithm 1 returns true iff there is a refinement-typing derivation for F under Γ and $\sigma_\star$.*

Proof. The algorithm directly mirrors the inference rules of Definition 5.1: each branch corresponds to one rule, the recursion follows the

structural premises, and the per-stage refinement check at Compose nodes implements the conjunction of Theorem 5.2. Correctness then follows from Theorems 6.1 and 6.2. $\square$ $\square$

7.3 The role of the bijection Φ

Proposition 7.4 (Label representation cost). *Under the bijection $\Phi : \mathbb{N}^+ \rightarrow \mathcal{L}$ from Theorem 3.3, each partition label is representable as a single positive integer; label comparison, hashing, and ordering are constant-time operations.*

Proof. By Theorem 3.3, Φ is a constructive bijection. Comparison and hashing of integers are standard $\mathcal{O}(1)$ operations on bounded-precision arithmetic. Ordering is the natural integer order. $\square$ $\square$

Corollary 7.5 (Selection-rule evaluation is constant-time). *Under the bijection Φ , each of the selection rules in Example 4.5 evaluates in $\mathcal{O}(1)$ time on a transition (λ_1, λ_2) .*

Proof. Each rule reduces to integer subtraction and bounded-range membership tests (e.g. “is $\ell_2 - \ell_1 \in \{-1, +1\}$ ”), both $\mathcal{O}(1)$. $\square$ $\square$

Together, Theorem 7.1, Proposition 7.3, and Corollary 7.5 give us a complete static-time enforcement mechanism: any vaHera fragment whose Compose chain implies a forbidden transition is rejected at compile time, in linear time in the fragment size.

Part V

Worked Example

8 The $\ell = 0 \rightarrow \ell = 2$ Rejection

We illustrate Theorem 6.3 on a small but representative example.

8.1 Setup

Fix the selection-rule registry $\sigma_\star = \{\sigma_{\text{E1}}, \sigma_{\text{spin}}\}$ from Example 4.5. We register two operations:

$$\begin{aligned} \mathcal{O}_1 : \{y : \mathcal{L} \mid y = (n_1, 0, 0, +\tfrac{1}{2})\} \rightarrow \\ \{x : \mathcal{L} \mid \sigma_{\text{E1}}((n_1, 0, 0, +\tfrac{1}{2}), x) \\ \wedge \sigma_{\text{spin}}((n_1, 0, 0, +\tfrac{1}{2}), x)\} \\ \mathcal{O}_2 : \{y : \mathcal{L} \mid y = (n_2, 2, 0, +\tfrac{1}{2})\} \rightarrow \\ \{x : \mathcal{L} \mid \sigma_{\text{E1}}((n_2, 2, 0, +\tfrac{1}{2}), x) \\ \wedge \sigma_{\text{spin}}((n_2, 2, 0, +\tfrac{1}{2}), x)\} \end{aligned}$$

The first operation takes an $\ell = 0$ source and produces any output reachable from it under E1 and spin-conserving rules. The second operation takes an $\ell = 2$ source and produces any reachable output. We attempt to compose them.

8.2 Direct composition: rejected

Consider the fragment

$$F = \text{Compose}(\text{Literal}((1, 0, 0, +\tfrac{1}{2})), \text{Call}(\mathcal{O}_2, \emptyset)).$$

The Compose chain threads the literal $\lambda_0 = (1, 0, 0, +\tfrac{1}{2})$ into the input of $\mathcal{O}_2$. The input refinement of $\mathcal{O}_2$ requires $\lambda_0 = (n_2, 2, 0, +\tfrac{1}{2})$; in particular, $\pi_\ell(\lambda_0) = 2$. But $\pi_\ell((1, 0, 0, +\tfrac{1}{2})) = 0$. The input refinement is violated, and by (R-CALL), the typing judgement fails. typecheck rejects F .

8.3 Through-route composition: also rejected

Consider the fragment

$$F' = \text{Compose}(\text{Literal}((1, 0, 0, +\tfrac{1}{2})), \text{Call}(\mathcal{O}_1, \emptyset), \text{Call}(\mathcal{O}_2, \emptyset)).$$

The first stage is typed at $\{(1, 0, 0, +\tfrac{1}{2})\}$ (a singleton refinement). The second stage, $\mathcal{O}_1$, accepts this input and outputs a label λ_1 satisfying $\sigma_{\text{E1}}(\lambda_0, \lambda_1)$ and $\sigma_{\text{spin}}(\lambda_0, \lambda_1)$. Reading off:

$$\pi_\ell(\lambda_1) - 0 \in \{-1, +1\} \implies \pi_\ell(\lambda_1) \in \{-1, +1\} \cap \mathbb{N} = \{1\},$$

forcing $\pi_\ell(\lambda_1) = 1$.

The third stage, $\mathcal{O}_2$, requires its input to satisfy $\pi_\ell = 2$. But the output of $\mathcal{O}_1$ has $\pi_\ell = 1$. The carry-threading transition $\lambda_1 \rightarrow \lambda_2$ feeds λ_1 with $\pi_\ell = 1$ into $\mathcal{O}_2$'s input refinement, which demands $\pi_\ell = 2$. This is a violation: (R-COMP) fails, and typecheck rejects F' .

8.4 Significance

The two rejections are structurally distinct—one fails at the Call stage, the other at the Compose threading—but both arise from the same underlying principle: the refinement type of the composed fragment is uninhabited. No physical $\ell = 0 \rightarrow \ell = 2$ single-photon transition exists under E1 selection rules, and no vaHera fragment can express it as a well-typed expression. The rejection is structural: there is no possible runtime that produces the implied transition, because no fragment can be written that implies it without violating the type system.

This is the central content of Theorem 6.3: forbidden transitions are not rare; they are type errors.

Part VI

Connections to Applications

9 Applications across the Symbolic-Physics Substrate

The refinement-type system specified here is parametric in the partition-coordinate structure and the selection-rule registry. We outline how the system applies as a foundational substrate for several downstream developments.

9.1 Bounded Hamiltonian Systems

The partition coordinates (n, ℓ, m, s) are the labels of bounded three-dimensional oscillatory systems, derived from first principles in Sachikonye (2026a). Every transition between such labels in a bounded Hamiltonian system must respect the closure constraints; the selection-rule registry $\sigma_\star$ encodes precisely those constraints. The present type system thereby provides the symbolic vocabulary for stating any bounded-Hamiltonian transition: a fragment is well-typed if and only if every transition it implies is admissible under the closure structure.

9.2 Mass-Spectrometric Trajectories

Ion trajectories in mass-spectrometric instruments—Orbitrap, FT-ICR, TOF,

quadrupole—are governed by a partition Lagrangian whose Euler-Lagrange equation reduces to the Lorentz force law (Sachikonye, 2026d). The state space of trajectories is parametrised by partition coordinates plus inertia and charge, and the trajectory’s evolution respects a set of motion-derived selection rules. Under the present framework, ion trajectories become well-typed vaHera fragments over the partition-label type, with the motion-derived rules as σ_* .

9.3 Temporal Programming and Cell Registries

The temporal-programming Tempus calculus (Sachikonye, 2026e) defines a cell registry that classifies timing deviations into operationally-defined cells. The cells are compile-time-immutable, partitioning a ΔP -space into action-equivalent regions. Under our framework, each cell becomes a refinement-typed operation: the cell’s bounds are the input refinement, the dispatched action is the operation body, and the cell-disjointness invariant is enforced as a type-system property. Selection rules over cell-to-action transitions become first-class predicates rather than hand-coded checks.

9.4 Real-Time Event-Selection Systems

In high-throughput event-selection systems—trigger systems for high-energy experiments—each detector subsystem registers as a refined operation whose output Value carries one partition coordinate of the detected particle. The full classification of a particle is a Compose chain over the subsystems, and the per-stage selection rules enforce the physical admissibility of the combined classification. Backgrounds whose hypothesised classification violates any selection rule fail typecheck before dispatch, requiring no runtime observation.

9.5 Three-Route Verification

The Three-Route Equivalence Theorem of computational operations (Sachikonye, 2026b) asserts that any partition-state invariant admits three independent computational measurements that agree to within a closed-form bound. Under the present type system, each route becomes a refined

operation, all sharing the same output type but with distinct internal computations. Verification reduces to running all three operations and checking that the outputs lie in the same refinement equivalence class—a type-level check rather than a numerical comparison.

Part VII Discussion

10 Comparison with Existing Refinement-Type Systems

The refinement-type literature contains several mature systems. We comment on the relationship of the present development to each.

10.1 Liquid Types

The Liquid Types system of Rondon et al. (2008) establishes refinement-type inference via predicate abstraction: refinements are SMT-decidable predicates over base types, and refinement inference is reduced to Horn-clause solving. Our system is restricted to the partition-coordinate domain, where the predicates are simple arithmetic comparisons; we obtain decidability in linear time without invoking SMT machinery. The trade-off is generality: Liquid Types apply to arbitrary base types, while ours is specialised to $\mathcal{L}$.

10.2 Refinement Types for Haskell

The system of Vazou et al. (2014) extends Liquid Types to a real-world functional language. The treatment of dependent function types and the encoding of inductive datatypes parallel our treatment of refined operations and Compose chains. We differ chiefly in the binary-predicate generalisation (4.2), which is not standard in the Haskell setting; binary predicates appear in dependent-type systems (Coquand and Huet, 1988; Norell, 2007) but in fully general form, without our compositionality lemma’s special-case efficiency.

10.3 ML-Style Refinements

Freeman and Pfenning’s refinement types for ML (Freeman and Pfenning, 1991) introduced regular tree types as refinements of inductive types. The Compose chain corresponds, structurally, to

a tree-shaped value flow, and our compositionality lemma is a strengthening of the tree-product property they establish. The forbidden-transition theorem is, in their language, a statement that the regular tree language defined by uninhabited refinements is empty.

10.4 Dependent Types

Coquand and Huet’s Calculus of Constructions (Coquand and Huet, 1988) and its successors (Agda (Norell, 2007), Lean (de Moura et al., 2015), Coq (Bertot and Castéran, 2004)) admit selection-rule predicates as dependent function types directly. Our framework can be viewed as a restricted, decidable fragment of such a dependent system, in which the predicate language is constrained to ensure the linear-time complexity of Theorem 7.1.

10.5 Hoare Logic

The Hoare-logic tradition (Hoare, 1969; Dijkstra, 1975) encodes pre- and post-conditions as assertions over program states. Our refined operations, with input refinement ϕ^{in} and output refinement ϕ^{out} , are Hoare triples specialised to vaHera dispatch. The compositionality lemma is the sequential composition rule of Hoare logic, with the carry-threading providing the implicit state.

11 Limitations and Open Questions

11.1 Predicate language

Our predicate language is restricted to constant-time computable Boolean functions on partition labels. More expressive predicates—second-order quantification, transitive closures, fixpoint computations—are outside the scope of the linear-time decidability result. Extending the framework to richer predicate logics would require a corresponding relaxation of the complexity bound.

11.2 Hole substitution

The treatment of Holes in Definition 2.7 assumes that all Holes have been resolved before typecheck runs. A more refined treatment would address Hole-resolution under refinement: given a fragment with unresolved Holes, what is the most

general refinement type compatible with all possible Hole substitutions? This question generalises principal-typing for parametric polymorphism (Milner, 1978; Damas and Milner, 1982) and is left for future work.

11.3 Operation registration

Operations are registered in a global registry $\mathcal{R}^+$. In a distributed setting, multiple registries may coexist with different selection-rule registries. Reconciling fragments typed against different registries—particularly the question of when a fragment well-typed under $\mathcal{R}_A^+$ remains well-typed under $\mathcal{R}_B^+$ —is a federation question that the present framework does not address.

11.4 Selection-rule learning

We treat the selection-rule registry σ_* as fixed and externally provided. The inverse problem—given a corpus of accepted transitions, infer the minimal σ_* —is an automated specification synthesis problem; we do not pursue it here.

12 Conclusion

We have introduced a refinement-type system over the vaHera AST in which partition coordinates are first-class types and selection rules are first-class refinement predicates. The system rests on three principal results: the Compositionality Lemma (Theorem 5.2), which establishes that refinements compose under Compose via conjunction; the Forbidden-Transition Theorem (Theorem 6.3), which establishes that selection-rule-violating fragments are rejected by typecheck at static time; and the Decidability Theorem (Theorem 7.1), which establishes that the joint judgement is decidable in linear time in the fragment size.

These results provide the symbolic substrate over which physical derivations expressing transitions between partition labels can be written and statically verified. The substrate is sufficient to specify any selection rule expressible as a computable binary predicate, and the resulting type discipline rejects every fragment whose composition chain implies a forbidden transition—without runtime observation, without probabilistic argument, and without recourse to external verification machinery.

The framework is foundational: it does not specify which partition coordinates are used, which selection rules are registered, or which operations populate the registry. These choices are downstream, made by the applications that build on the substrate. What the framework guarantees is that, once those choices are made, the resulting symbolic representation has a statically-decidable well-formedness criterion, and that the criterion is structural rather than probabilistic.

The downstream applications outlined in Section 9—bounded Hamiltonian dynamics, mass-spectrometric trajectories, temporal-programming cell registries, real-time event-selection systems, and three-route verification—each adopt the substrate by registering an appropriate partition-coordinate domain and selection-rule registry. None of them re-implements the type-checking machinery; each cites the present development for its symbolic foundations and contributes its domain-specific operations on top.

References

- Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer, 2004.
- Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2-3):95–120, 1988.
- Luis Damas and Robin Milner. Principal type-schemes for functional programs. In *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 207–212. ACM, 1982.
- Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *Automated Deduction — CADE-25*, volume 9195 of *Lecture Notes in Computer Science*, pages 378–388. Springer, 2015.
- Edsger W. Dijkstra. Guarded commands, non-determinacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975.
- Tim Freeman and Frank Pfenning. Refinement types for ML. In *Proceedings of the ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation (PLDI)*, pages 268–277, Toronto, Ontario, Canada, 1991. ACM.
- Roger Hindley. The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146:29–60, 1969.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.
- Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory*. PhD thesis, Chalmers University of Technology, 2007.
- Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. Liquid types. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 159–169, Tucson, AZ, USA, 2008. ACM.
- Kundai Farai Sachikonye. On the bijective correspondence between integer sequences and partition states in bounded hamiltonian systems: Counting, monotonicity, and causal succession. Monograph, Volume I, 2026a.
- Kundai Farai Sachikonye. Computational operations equivalence: A single-quantity bookkeeping framework for kernel operations. Buhera Project, Volume IV, 2026b.
- Kundai Farai Sachikonye. Integrating with the purpose framework: The stability contract. Technical specification, Buhera Project, 2026c. Available as long-grass/integration.md.
- Kundai Farai Sachikonye. Ion trajectory completion mechanism: Partition lagrangian, mass spectrometry, and mass from three routes. Monograph, Volume II, 2026d.
- Kundai Farai Sachikonye. Temporal programming: A cell-based paradigm for oscillator-relative distributed computation. Monograph, Volume III, 2026e.

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 269–282, Gothenburg, Sweden, 2014. ACM.